

Problem 1: Floating point—mind the gap

1.

In a 32-bit IEEE floating point number, what is the smallest number greater than 10 that can be exactly represented? What is the gap between that number and 10? Briefly justify your answers.

The decimal number 10 is represented in binary as:

$$10_{10} = 1010_2$$

We express this in the form:

$$10_{10} = 1.010_2 \times 10^3$$

To find the smallest number greater than 10, we take the maximum number of trailing zeros and place a single 1 at the very end. This ensures it is the next representable number larger than 10.

Thus, in IEEE standard, this number is represented as:

0 1000 0010 010 0000 0000 0000 0000 0001

This ensures that the number remains as close to 10 as possible while being strictly greater than it. This is equal to $1.010000000000000000000001_2 \times 10^3$. The gap between them is $1010_2 - 1010.00 \dots 1_2 = 2^{-20}$

2.

Assuming we are always rounding to the nearest value we can exactly represent (rounding down in the case of ties), what is the largest value we can add to 10 and get a result that is still 10?

For the last part, the largest value that can be added to 10 and still have it round to 10 is 2^{-21} . Near 10 in the 32-bit IEEE representation, the gap between consecutive representable numbers is 2^{-20} . When rounding to the nearest value (with ties rounding down), any value less than half of that gap—namely, 2^{-21} —won't push the total over the midpoint between 10 and the next representable number, which is $10 + 2^{-20}$. Thus, even when you add 2^{-21} to 10, the sum is still closer to 10 than to $10 + 2^{-20}$, ensuring that it rounds back to 10.

3.

Redo part 1) but for -17.125 rather than 10?.

The decimal number 17.5 is represented in binary as:

$$17.5_{10} = 10001.001_2$$

We express this in the form:

$$17.5_{10} = 1.0001001_2 \times 10^4$$

Doing the same thing as part 1 we can add the max amount of zeros and place a 1 and the end. Thus, we get:

1 1000 0011 000 1001 0000 0000 0000 0001

This is equal to $1.000100100000000000000001_2 \times 10^4$. Thus, the gap is 2^{-19}

4.

Briefly explain how the gap between floating point numbers might cause a programmer difficulties. In particular, consider a for loop where small values are being added to a larger number many thousands of times.

Floating-point numbers have non-uniform spacing; as numbers grow larger, the gap between them increases. This means that when a small value is repeatedly added to a much larger number—say, inside a for loop—the addition might not change the larger number at all because the increment is smaller than the gap between representable numbers. For example, if the increment is less than half the gap, it may be effectively “lost” due to rounding, and the running total may remain unchanged for many iterations. This can lead to unexpected behavior and accumulated errors in calculations, posing a significant challenge in numerical computations and simulations.

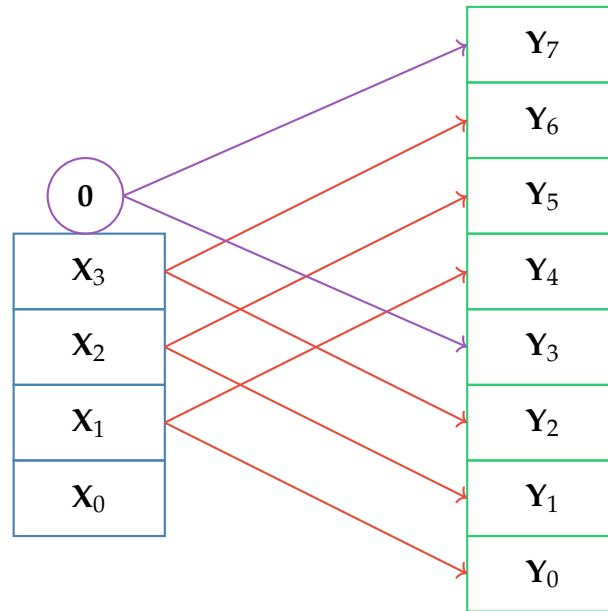
Problem 2: Linking

If a global variable is declared in a source file named `file1.c`, do accesses to it in `file1.c` need to go in the relocation table when linking? Why or why not? Briefly explain your answer.

No, they don't. When `file1.c` is compiled, the compiler already figures out the exact location of the global variable in that file. This means that any reference to the variable within `file1.c` already knows where to go, so there's no need for the linker to adjust these addresses later using the relocation table.

Problem 3: Multiplication-by-wire

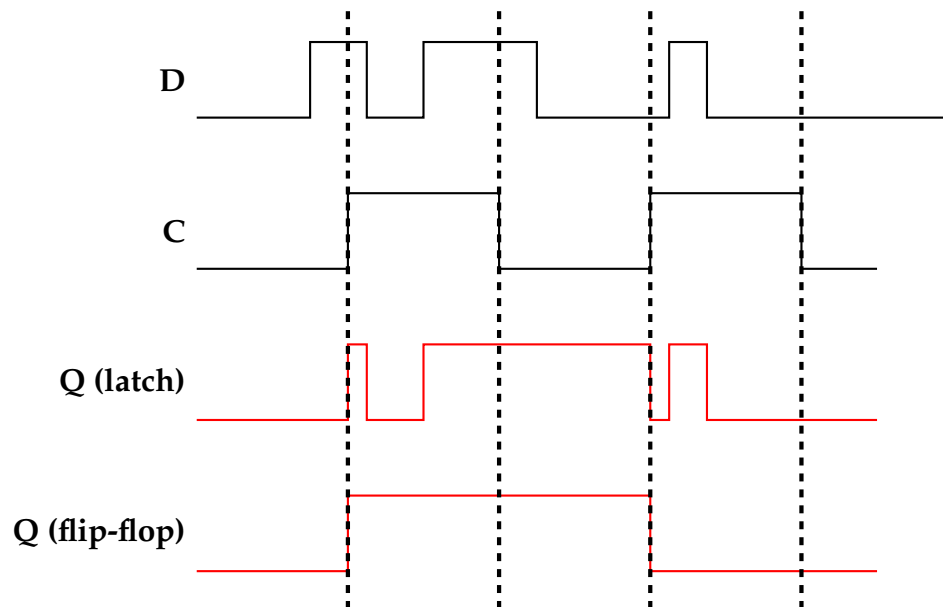
Say you have a 4-bit unsigned number $X[3 : 0]$ (so X_3 is the most significant bit, X_2 is the next most significant, etc.). Using only standard gates (AND, OR, NOT), and constant 0s and 1s, multiply $X[3 : 0]$ by 8.5 and output it as an 8-bit unsigned number $Y[7 : 0]$. Use as few gates as possible, even if it means you use no gates. You may assume that X is divisible by 2.



This approach works by breaking the multiplication by 8.5 into two separate steps: first multiplying by 8 and then by 0.5, before combining the results. Multiplying by 8 in binary is equivalent to shifting all bits left by three positions, which moves X_3 to Y_6 , X_2 to Y_5 , and X_1 to Y_4 , while X_0 , which is always 0 because X is given to be divisible by 2, shifts into Y_3 , which is also set to 0. Similarly, multiplying by 0.5 is equivalent to shifting all bits right by one position, moving X_3 to Y_2 , X_2 to Y_1 , and X_1 to Y_0 , with X_0 being dropped entirely. Since the results from these two shifts do not overlap in $Y[7 : 0]$, we can simply place the shifted values in their respective positions without needing to add or modify any bits. The remaining positions in Y that are not assigned by either shift—specifically Y_7 and Y_3 —are set to 0. Because this transformation is achieved entirely through bit shifting and direct wiring, there is no need for additional logic gates, making this an efficient and minimal implementation. The fact that X_0 is always 0 further simplifies the process, ensuring that no extra handling is needed for that bit during shifting operations.

Problem 4: Latches and flip-flops

Complete the timing diagrams for a D latch and D flip-flop given the provided inputs. The flip-flop is positive-edge triggered. For the latch we are using “C” for the “G” input (which is fairly common). Assume both the latch and the flip-flop have an initial value of zero



Problem 5: State machines**1.**

Consider a state machine implementation with 6 state bits. What is the maximum number of states you can have with such an implementation?

A state machine with 6 state bits can represent a maximum of $2^6 = 64$ unique states, as each bit can be either 0 or 1, resulting in 2^6 possible combinations.

2.

Say you have a state machine with 5 bits of input, 3 bits of output, and 24 states. How large would your control ROM need to be? Fill in the blanks below.

10 bits of address

8 bits data per address

A total of 2^{13} bits in the ROM.

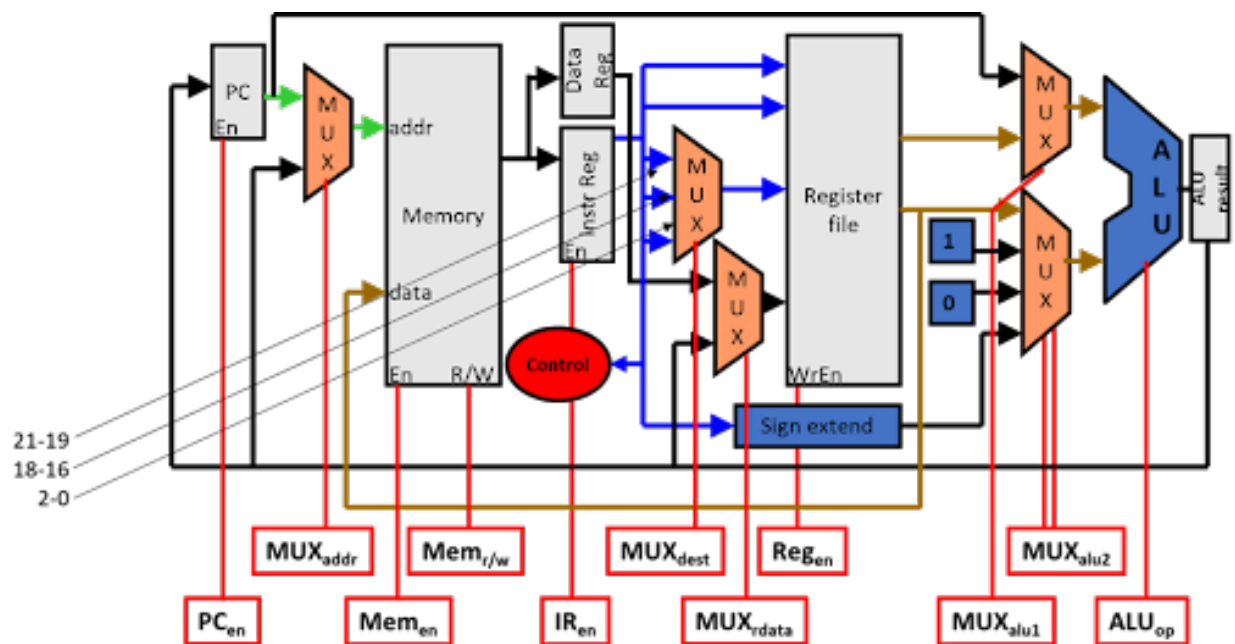
Problem 6: Incremental Improvement

In a moment of clarity, the EECS 370 staff realizes that the LC2K instruction set is rather limited in its capability to take advantage of the AI boom. To improve the situation, the EECS 370 staff decides to implement a new instruction: load from pointer (LFP). The semantics of the LFP instruction are provided below.

lfp regA regB

//REG[regA] = MEM[REG[regA]];

//REG[regB] = MEM[REG[regA]];



Cycle	PC (en)	MUX (addr)	Mem (en)	Mem (r/w)	IR (en)	MUX (dest)	MUX (rdata)	Reg (en)	MUX (alu1)	MUX (alu2)	ALU (op)
1	0	0	1	0	1	XX	X	0	0	01	0
2	1	X	0	X	0	XX	X	0	X	XX	X
3	0	X	0	X	0	XX	X	0	1	10	0
4	0	1	1	0	0	XX	X	0	X	XX	X
5	0	X	0	X	0	00	0	1	X	XX	X
6	0	X	0	X	0	01	0	1	X	XX	X

Problem 7: SC/MC Datapath Performance

1.

What is the clock period for the single-cycle processor if we only had to support add, halt, beq, and lw?

Since lw is the longest instruction the clock period would be 24ns.

2.

What is the clock period for the single-cycle processor if we support all LC2K instructions except jalr?

Well again since lw is the longest instruction the clock period would be 24ns.

3.

If we can decrease the delay for one of the operations by 25%, which operation should it be for our single-cycle datapath which supports all LC2K instructions except jalr? What is the clock period after improvement?

So lw is the longest at 24ns and sw is right behind at 23ns. If we decrease read memory we decrease every instruction by 2, and lw by 4ns, thus we get a clock speed of 21ns since sw is now the longest.

4.

If we can decrease the delay for one of the operations by 25%, which operation should it be for our multi-cycle datapath which supports all LC2K instructions except jalr? What is the clock period after improvement?

Since multi-cycle clock speed is based on the longest stage which is when sw writes to memory we can reduce write memory by 25% and get a new clock speed of 9ns since no other stage was close to 12ns.

Problem 8: Intro to Pipelining

a.

Fill in the timing graph below with the pipeline stage that each instruction is in at the start of each cycle. Recall that the stages are IF, ID, EX, MEM, WB. The first IF has been filled in for you.

Instruction \ Cycle #	0	1	2	3	4	5	6	7	8
add 4 7 1	IF	ID	EX	MEM	WB				
lw 0 6 1		IF	ID	EX	MEM	WB			
nor 5 3 3			IF	ID	EX	MEM	WB		
sw 0 2 8				IF	ID	EX	MEM	WB	
halt					IF	ID	EX	MEM	WB

b.

Fill in the contents of the pipeline registers at the point in time between cycle A and B.

Cycle 0-1:

IF/ID		ID/EX		EX/MEM		MEM/WB	
Opcode:	add	Opcode:	noop	Opcode:	noop	Opcode:	noop
PC Plus 1:	1	PC Plus 1:	X	aluResult:	X	writeDate:	X
		regA val:	X				
		regB val:	X	regB val:	X		
		offset:	X				

Cycle 2-3:

IF/ID		ID/EX		EX/MEM		MEM/WB	
Opcode:	nor	Opcode:	lw	Opcode:	add	Opcode:	noop
PC Plus 1:	3	PC Plus 1:	2	aluResult:	2	writeDate:	X
		regA val:	-27				
		regB val:	6	regB val:	X		
		offset:	1				

Cycle 4-5:

IF/ID		ID/EX		EX/MEM		MEM/WB	
Opcode:	halt	Opcode:	sw	Opcode:	nor	Opcode:	lw
PC Plus 1:	5	PC Plus 1:	4	aluResult:	0	writeDate:	??
		regA val:	4				
		regB val:	-4	regB val:	X		
		offset:	8				

Cycle 5-6:

IF/ID		ID/EX		EX/MEM		MEM/WB	
Opcode:	X	Opcode:	halt	Opcode:	sw	Opcode:	nor
PC Plus 1:	X	PC Plus 1:	X	aluResult:	X	writeDate:	0
		regA val:	X				
		regB val:	X	regB val:	-4		
		offset:	X				